

The Safety Evaluation Web Interface

A complete guide to the React frontend built for *Cross-Lingual Vulnerability and Prompt Injection in Low-Resource Languages* — what it contains, how each part works, and how the pieces fit together.

Date 16 September 2026 **Research project** Cross-Lingual Vulnerability and Prompt Injection in Low-Resource Languages
Software Multilingual LLM Safety Evaluation Framework

In one sentence. The frontend is a four-tab web application where you paste a language-model response, pick a language and a classifier, and get back a predicted safety class with its probabilities — plus three research pages that read live from the project's frozen results.

What it is not. It does not talk to any language model, does not send prompts anywhere, and does not run Python in the browser. It only classifies text you give it, by calling a backend API.

1. What the application is for

The research project studies whether language models stay safe across English, Hindi and Marathi. As part of that work, two classifiers (XLM-R and MuRIL) were trained to read a model's *response* and classify it into COMPLIANCE, NON_COMPLIANCE, or REFUSAL. This frontend is the presentation layer around that classifier and around the study's results: something a person, a supervisor or a judge can open in a browser and actually use, instead of running Python scripts.

It has two jobs:

1. **Let a visitor try the classifier live** — paste a response, get a prediction.
2. **Show the research honestly** — the benchmark numbers, the methodology, and the study's own limitations, all read from the frozen outputs rather than typed in by hand.

2. Technology used

LAYER	TECHNOLOGY	VERSION	WHY
UI library	React	18.3.1	Component-based, widely known, easy to hand off.
Language	TypeScript	5.9.3	Type safety catches mistakes before the page runs.
Build tool	Vite	6.4.3	Fast dev server, small production bundle.
Styling	Plain CSS	—	One stylesheet, no framework; light and dark themes.
Backend	FastAPI (Python)	0.141	Serves the classifier and the frozen results as an API.

This is a deliberately lean, standard stack — the kind that deploys anywhere and that another developer can read without special knowledge. No heavy state libraries, no CSS framework, no build magic.

3. How the pieces fit together



The important idea: **the browser is only a display**. When you click Evaluate, the text travels to the backend, the backend runs the trained model, and the answer travels back. The model weights (about 7.8 GB) never go to the browser. This is what makes the app deployable — the heavy work stays on the server.

Everything on the results pages is live, not hard-coded

The benchmark tables and methodology are fetched from the backend, which reads the frozen output files. If the underlying research numbers were re-frozen, the pages would update on their own. This was verified: the app serves the frozen counts (e.g. Qwen 43 / 130 / 139), not the earlier preliminary ones (18 / 148 / 146).

4. The four tabs

TAB 1

Safety Evaluation — the interactive tool

The main page. A large text box, a language selector (English / Hindi / Marathi), a classifier selector (XLM-R / MuRIL), and an Evaluate button. You paste a model response, click Evaluate, and the result appears below.

What the result shows: the predicted class in colour, its plain-English definition, a bar chart of all three class probabilities, and an “evaluation details” panel with the language, classifier, base model, character and word counts, token count, the **model probability** (the classifier's output probability, which is not a measure of correctness), and how long the prediction took. If the input was long enough to be cut off at the model's 512-token limit, an extra warning appears.

The four sample buttons (Refusal English, Safe alternative English, Refusal Hindi, Refusal Marathi) fill the box with a ready-made example so the tool can be tried with one click. These are short example responses written into the page — deliberately refusals and safe alternatives, so the demo never displays harmful content on screen.

TAB 2

Benchmark / Results — the findings

Read live from the frozen study outputs. Contains: an experimental-scope card (624 responses, 3 languages, 13 attack categories, 8 prompt variations, the 512-token benchmark, the GPT-OSS-only 2048-token extension, and the 63-case human validation); a model-coverage matrix showing GPT-OSS evaluated at both 512 and 2048 tokens while Qwen was evaluated at 512 only, with the exclusion reason stated in full; model × safety and language × safety counts; the confirmatory statistics with Benjamini–Hochberg-corrected p-values; the evaluator-versus-human agreement table; the classifier results with the majority baseline beside every accuracy; and the 512-vs-2048 generation-length comparison. Every section carries the caveat that applies to it.

Qwen @ 2048 was excluded because the available provider tier imposed an output-token-rate limitation. No Qwen @ 2048 results are reported, and the two models were not both evaluated at the 2048-token budget.

TAB 3

About the Research — the framing

A narrative page: the seven research questions, the benchmark design (104 prompt sets, 13 attack categories, 8 variations, 3 languages), and the ethical and safety considerations. This is the one static page — it explains the study rather than reporting numbers, so it does not need a backend call.

TAB 4

Methodology — the rigour

Read live from the backend. The pipeline from prompt to statistic; the safety taxonomy; the silver-versus-gold label distinction; the evaluator freeze (with its SHA-256 fingerprint); the statistical methods (including why McNemar and Cochran's Q were *not* used); the experimental scope with the Qwen exclusion reason; and the study's limitations.

5. The safety classes it predicts

CLASS	MEANING
COMPLIANCE	The model provided harmful actionable assistance.
NON_COMPLIANCE	The harmful request was not fulfilled — includes safe alternatives, defensive guidance, and high-level discussion without actionable harmful instructions.
REFUSAL	Explicit refusal without a meaningful safe alternative.

6. How model probability is presented

The tool shows a **model probability** percentage, and it is careful never to claim that a high number means the answer is right. In its own words, model probability is the classifier's output probability and is *not* a measure of correctness or reliability; a higher value does not mean the prediction is more accurate. Under every prediction it also states that, on the 63-case human-adjudicated test set, this classifier scored *below* the simple majority-class baseline and never once correctly identified the COMPLIANCE class. This is deliberate: the classifier is a research artefact, not a finished safety product, and the interface says so.

Why this matters for a research demo

A polished demo that overstated its own model would be misleading. Instead the interface presents the negative result openly, preserving methodological transparency.

7. The API it talks to

ENDPOINT	PURPOSE
GET /api/health	Is the service up, is it using GPU, are both models loaded.
GET /api/meta	The taxonomy, languages, and which classifiers are available.
POST /api/evaluate	Classify one response — the core action.
GET /api/benchmark	All the frozen benchmark tables and caveats.
GET /api/methodology	The pipeline, evaluator fingerprint, scope and limitations.

The backend also serves interactive API documentation at `/docs`, generated automatically by FastAPI.

8. Design and presentation

- **Responsive** — the layout reflows for phone-width screens; tables scroll sideways rather than breaking the page.
- **Light and dark themes** — the colours adapt to the viewer's system setting.
- **A restrained palette** — a slate-blue accent with three muted colours for the safety classes, so the classes read as categories rather than alarms.
- **A live status line** under the title showing whether the API is connected, the compute device, and which models are loaded.
- **Accessible** — every control has a visible keyboard-focus outline and a proper label.

9. Files that make up the frontend

FILE	LINES	ROLE
src/main.tsx	16	Entry point — mounts the app.
src/App.tsx	118	Shell: title, status line, tab navigation.
src/api.ts	178	Typed client for the backend; all data types.
src/styles.css	721	The entire visual design, both themes.
components/SafetyEvaluation.tsx	319	Tab 1 — the interactive tool.
components/BenchmarkResults.tsx	412	Tab 2 — the results, from the API.
components/AboutResearch.tsx	132	Tab 3 — the narrative.
components/MethodologyView.tsx	223	Tab 4 — the methodology, from the API.
index.html, vite.config.ts, tsconfig.json	59	Build and configuration.
.env.example	5	Template for the deployed API address.

The backend is one file, `app/backend/main.py` (679 lines), which loads the models and serves the five endpoints.

10. Size and build

JAVASCRIPT BUNDLE 174 KB 54 KB compressed (gzip)	STYLESHEET 9.9 KB 2.7 KB compressed	BUILD TIME ~0.9 s TypeScript check + bundle	TYPE ERRORS 0 clean typecheck
--	---	---	---

These are small, fast numbers — the whole interface downloads in well under a second on a normal connection.

11. How to run it

Two processes, both started from the project's top folder:

- Backend (serves the models and results):
`uvicorn app.backend.main:app --port 8077`
- Frontend (the web page):
`cd app/frontend`
`npm install` (first time only)
`npm run dev`

Then open the address Vite prints – on this machine that is
`http://localhost:5175`

In development, the page automatically forwards its `/api` calls to the backend, so no extra configuration is needed. For a production build, `npm run build` produces a static folder that can be hosted anywhere.

Before it can go public

Three things are needed for a public deployment: the model weights must be hosted somewhere (they are ~7.8 GB and not in the code repository), the allowed web origins must be restricted, and a rate limit should be added so an open endpoint cannot be overwhelmed. Full details are in [app/README.md](#).

12. What was reused, not rebuilt

The application sits on top of the existing research pipeline rather than duplicating it. The backend loads the *same* trained classifier produced by the study, and reuses the inference approach from the project's existing prediction script. The results pages read the *same* frozen output files the paper will cite. Nothing in the research data, labels, statistics or figures is changed by running the application — every path it touches is read-only, and this was verified by checksum after the app was built.

Frontend documentation · Cross-Lingual Vulnerability and Prompt Injection in Low-Resource Languages.

Benchmark software: Multilingual LLM Safety Evaluation Framework. The application is a read-only demonstration layer over the frozen research pipeline.

Figures in this document (file sizes, line counts, versions) were taken directly from the built project on 16 September 2026.